

Apskaita5 ERP Modernization — Case Study

Oct 5, 2026 · @Kirill Fuchs

Executive summary

Recommendation: rebuild Apskaita5 as one multi-tenant cloud product using AI-assisted modernisation, and move customers one at a time behind a financial-equivalence safety net.

Business goal	What the plan delivers
Multi-tenant cloud product in 12 months	First customers in month 5; all nine modules live by month 9
Run cost per customer down 45%	Down 55% per migrated customer at month 12; down 59% across all customers once legacy is switched off (month 20)
Statutory turnaround under 3 weeks	20 days at most in the cloud; legacy customers kept inside the 6-week legal window
70% of customers migrated	2,970 (70.7%) by month 12; everyone by month 18
Zero financial regression	Every ledger, VAT and payroll number matches to the cent before a customer moves

Why it hurts today. Every customer has its own database, so cost grows with every customer. Every tax-form version is a copied class, built into 310 forks. And 522,000 lines of 2006-era code have no tests.

Five moves:

1. **One platform:** a modular monolith on pooled PostgreSQL, isolated by tenant, in cells of about 600 customers.
2. **Statutory rules as data:** signed, effective-dated rule packs that ship without a code release.
3. **No more forks:** each customization becomes a setting, a template or an out-of-process extension.
4. **AI at four layers (L0 to L3):** AI drafts, tools verify, people sign.
5. **Overnight moves in waves,** with a rehearsed way back.

Money. About 3.95M EUR over two years. Savings of 2.2M EUR a year from year 3, payback in month 39, and 3.96M EUR net cash over five years.

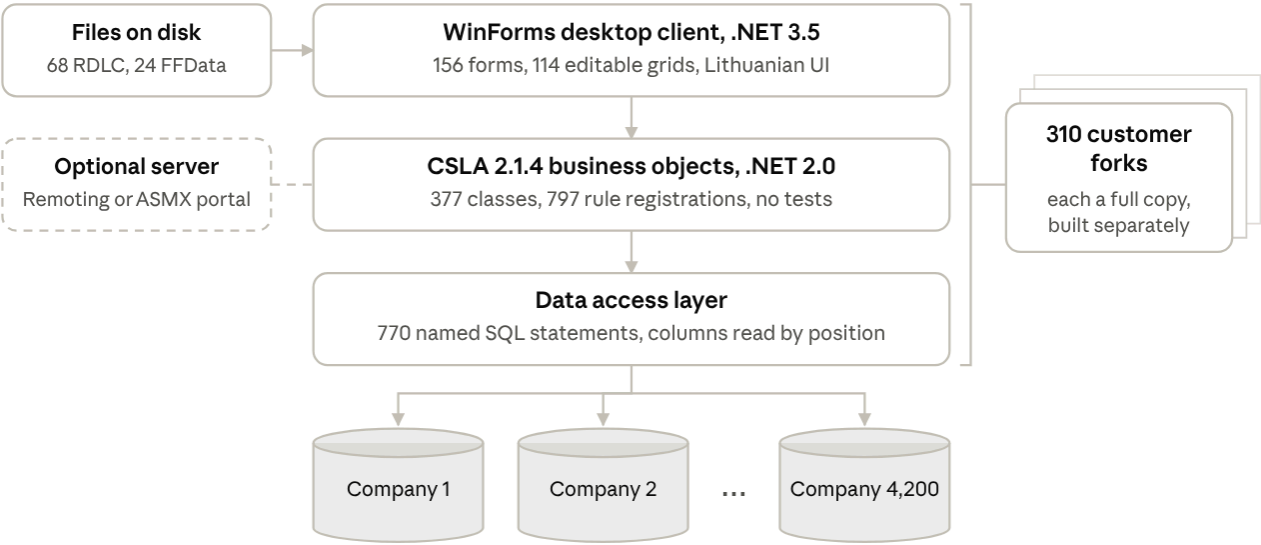
Decision requested: fund an eight-week foundation phase. Full rollout goes ahead only if gate P1 passes: assumptions confirmed, the rule inventory at 95% precision, and the legacy test harness running.

Current-state assessment

The reference code is 522,000 lines of VB.NET on .NET Framework 2.0 and 3.5 and CSLA 2.1.4, a 2006-era stack with no automated tests. Two things make it tractable. Business logic is separate from the UI, and all SQL sits in one named dictionary.

Today: one desktop stack, copied 310 times, over 4,200 separate databases

Current architecture as found in the reference code; dashed parts are optional



One MySQL database per company: 67 tables, money in DOUBLE, no foreign keys

What we found in the code	What it means for the plan
Money is stored as floating point, and five copies of the rounding code disagree, on about 1 in 550 VAT calculations	Equivalence is tested to the cent under a written tolerance policy
No foreign keys; columns are read by position	Relationships are inferred, and every database is profiled before it moves
No schema version; customers upgrade their own database by hand	Pre-flight brings all 4,200 databases to one structure first
Each tax-form version is a copied class, with dates and 20 year-specific rules hardcoded	The root of the 10-to-14-week delay; it becomes data
Every company types in its own tax rates	Central rule packs remove a whole class of errors

What we found in the code	What it means for the plan
Desktop users can write straight to MySQL	Writers are locked out at cutover, not just asked to stop
Logic is separate from the UI; SQL sits in one dictionary	Legacy runs headless as the test oracle, and the AI inventory starts from a complete map
A database password ships in plain text	Rotate it now

Assumptions

Thirteen facts the brief leaves open, why each was needed, and what it changed. All are confirmed in the first eight weeks, at gate P1.

ID	Assumption	Why needed	Design impact
A1	Target cloud is Microsoft Azure, two EU regions; AWS works equally.	No cloud named; .NET stack; GDPR.	Managed PostgreSQL, Container Apps, EU-hosted AI.
A2	The 3.1M EUR covers hosting, not salaries; a support ticket costs 12 EUR.	Baselines for the cost and support targets.	738 EUR per customer today; target 406 EUR or less.
A3	Median database under 2 GB; largest 5% under 50 GB.	Size decides pooling and migration speed.	Pooled by default; 100 migrations a night.
A4	About 9 users per company; many are bureau accountants serving many companies.	38,000 users across 4,200 companies.	One login across companies; bureaus move together.
A5	Lithuania first; Latvia and Estonia later.	The code targets VMI and Sodra.	Jurisdiction is part of every rule pack.
A6	About 30 engineers in-house, four with the tribal knowledge; a delivery team of 32 is added for 12 months.	The brief gives no headcount.	Sizes the team plan and the value of freed capacity.

ID	Assumption	Why needed	Design impact
A7	Forks mostly change reports, fields, integrations and validations, not posting logic.	Collapsing forks depends on what they change.	Three extension tiers; misfits move in later waves.
A8	No software certification in Lithuania; records must stay readable for 10 years.	Decides the release gates and when legacy can retire.	Schema validation gates releases; legacy is archived, not deleted.
A9	Accountants accept a browser app if keyboard speed matches.	Decides whether a desktop wrapper is needed.	Keyboard-first web client with a speed-parity gate.
A10	"Zero regression" means ledger, VAT and payroll match legacy to the cent; legacy bugs are fixed only through a signed register.	The goal needs a testable definition.	Drives L3 testing and every migration gate.
A11	Statutory turnaround runs from official publication to production.	The KPI needs a start and an end.	Rules ship as data, outside code releases.
A12	Legacy business objects run without the desktop UI.	The test harness drives legacy code directly.	Legacy is the test oracle in L3.
A13	Hosted AI may process code and data under an EU agreement, with no training on customer data.	AI is used in delivery and in the product.	EU-hosted models, redaction, consent before fork analysis.

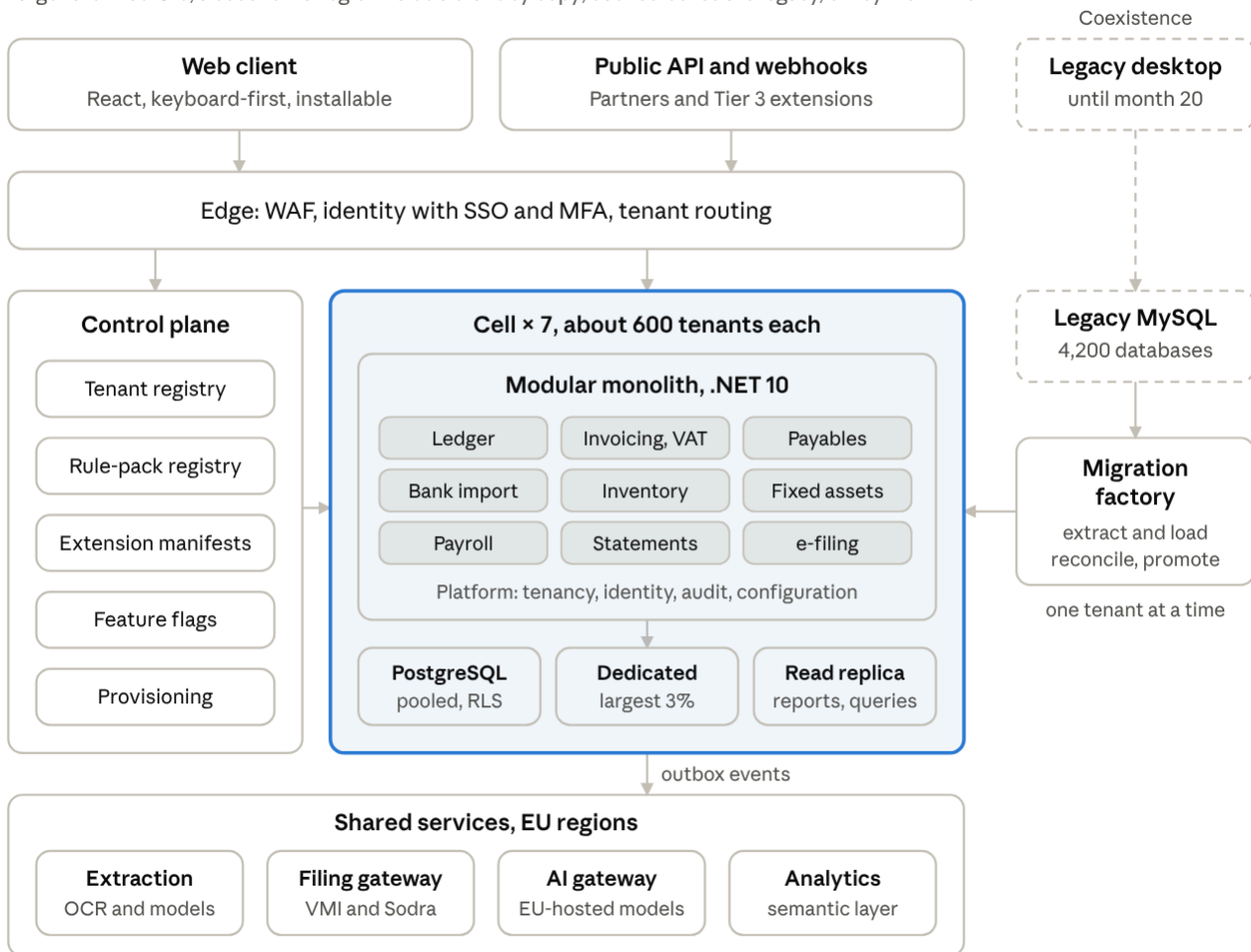
Target architecture

One code base for every customer, built from five parts:

- a modular monolith on .NET 10;
- pooled PostgreSQL in cells of about 600 tenants;
- a keyboard-first web client;
- statutory rules as data;
- three extension tiers instead of forks.

Every tenant runs one release inside pooled cells

Target architecture; a second EU region holds a standby copy; dashed boxes are legacy, off by month 20



Target architecture · clients, edge, control plane, cells, shared services, coexistence

Tenant isolation

- **One tenant is one set of books**, the same unit as one of today's 4,200 databases. Bureau users can belong to many tenants, but every request is bound to one.
- **Pooled by default**. Tenants in a cell share one PostgreSQL schema.
 - Row-level security filters every table. It is forced and the app role cannot bypass it. The tenant is set per transaction, and a request with no tenant is refused.
 - Every key includes the tenant, so no row can point into another tenant.
 - The largest 3 percent get a dedicated database running the same code.
- **Cells of about 600 tenants** cap the blast radius and are the unit of canary release. There are seven at full scale.
- **Isolation beyond the database**:
 - Queue messages, cache keys, document paths, search, exports and logs all carry the tenant.

- Support access is scoped and logged.
- Cross-tenant probes run every hour, and any leak blocks a release.

Modular monolith and financial core

- **Nine modules, one deployable.** Each module owns its own schema, and architecture tests guard the boundaries. Posting an invoice touches four modules in one transaction, so correctness never depends on a distributed system.
- **Events leave through a transactional outbox** to four separate workers: document extraction, the filing gateway, analytics and the migration factory.
- **Ledger rules:**
 - Posted entries never change; corrections are reversals.
 - Every posting carries a unique key, so a retry never posts twice.
 - Money is exact decimals, and accounting, tax and effective dates are kept separate.
 - Closing a period locks posting into it.

Web client for keyboard-heavy accountants

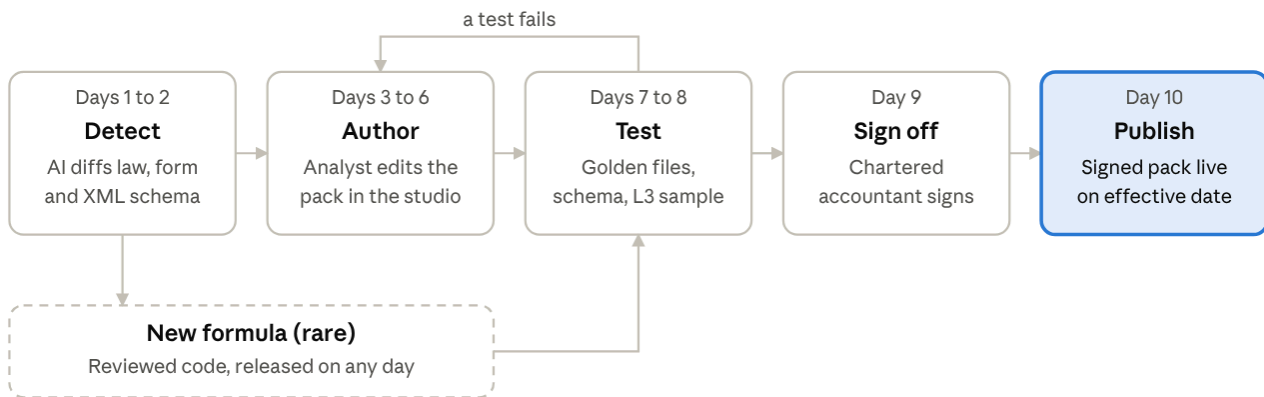
- **React and TypeScript with a spreadsheet-grade grid.** A classic key map copies today's habits: Insert and Delete for rows, Enter and Tab across cells, type-ahead pickers, and date shorthand such as "5" or "-5".
- **Speed parity is a release gate.** The 25 most frequent workflows are timed on legacy and web with power users. A module ships only when 90 percent are as fast or faster.

Statutory variability as configuration

- **Rule packs per jurisdiction.** Each pack holds effective-dated rates and thresholds, form mappings, rounding policies and golden test files. It is signed and released without a code deploy.
- **Formulas stay in reviewed code,** versioned with the pack. There is no scripting language, and no AI executes tax rules.
- **Every calculation records its pack version,** so history is never silently recalculated.
- **Legacy customers are covered too.** The same analysis and test files feed the ring-fenced legacy team, which ships each change within the 6-week legal window.

A statutory change ships as data in about 10 working days

Worst case 14 working days (20 calendar days), with no code release; legacy takes 10 to 14 weeks



Statutory change pipeline · 5 steps, budget in working days

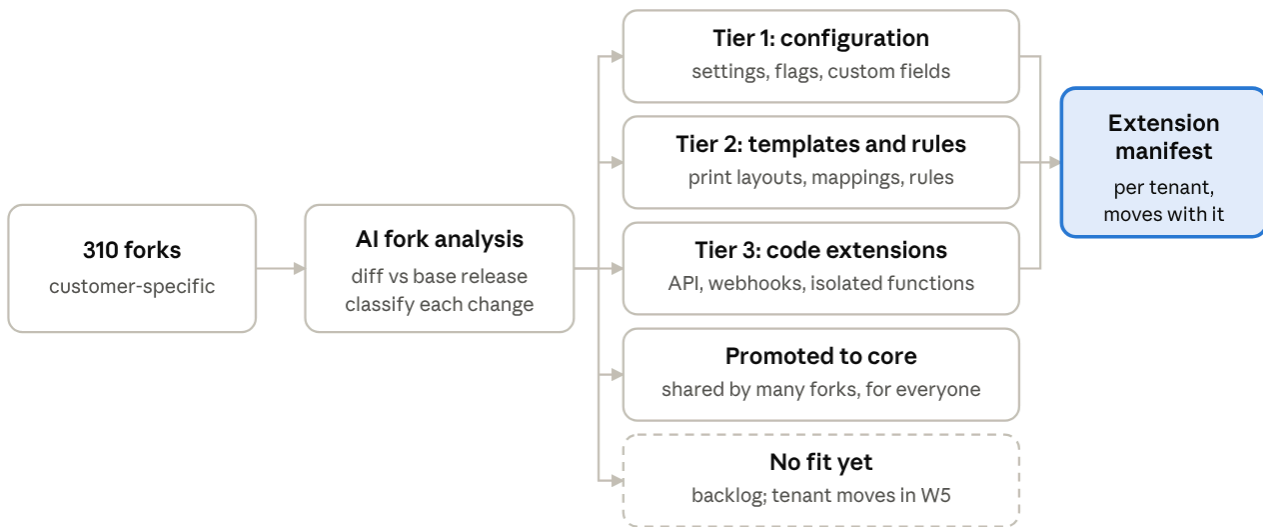
Most changes are new rates, form versions or schema fields, and they travel as data. A new formula ships in a code release, which can go out on any day, so it meets the same 20-day limit.

From 310 forks to extension points

- **Classify every change.** AI diffs each fork against the release it branched from and classifies every change. All 310 are classified by week 6.
- **Three tiers:** configuration; templates and rules; and out-of-process code extensions through the API and webhooks. Extensions act only through validated commands and never write to ledger tables.
- **Manifests, not forks.** Each tenant migrates with an extension manifest. A fork change that fits no tier becomes a product backlog item, and that tenant moves in a later wave.

Every fork change lands in a tier, in the core, or on the backlog

Fork consolidation; no tenant keeps a code fork in the cloud



Fork consolidation · 310 forks into three tiers, core, or backlog

Platform and service levels

Layer	Choice
Client	React, TypeScript, AG Grid
Application	.NET 10 (LTS), ASP.NET Core, C#
Data	Azure Database for PostgreSQL, zone-redundant, with a read replica per cell
Compute and messaging	Azure Container Apps, Service Bus
Identity and security	Entra External ID, Front Door with WAF, Key Vault
AI	Document Intelligence; Azure OpenAI in the EU Data Zone, behind a model gateway

Targets:

- 99.9 percent monthly availability, rising to 99.95 percent in filing week.
- A regional disaster loses at most 15 minutes of data, with service back within 4 hours.
- API responses under 300 ms.

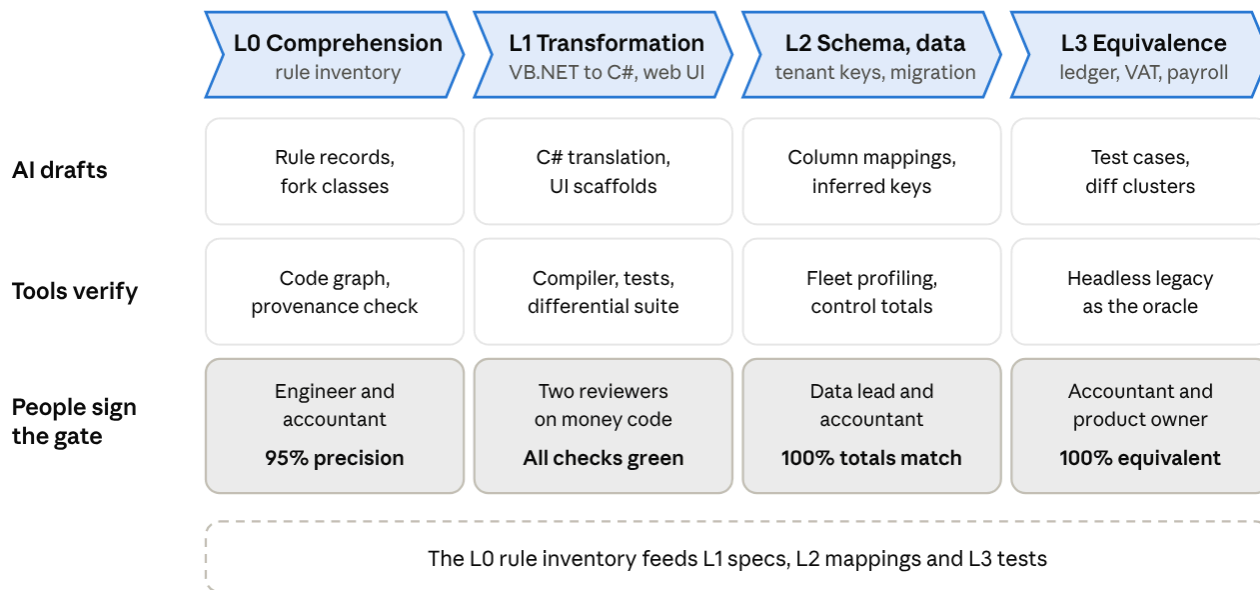
The full stack is in the Reference tab.

Layered AI in the modernisation lifecycle

At each of four layers, AI drafts, deterministic tools verify, and a named person signs a gate with a threshold. AI never moves financial data at run time and never approves its own work.

AI drafts, tools verify, people sign: the same pattern at every layer

Modernisation lifecycle; a layer's output moves on only after its gate passes



Layered AI · four layers, three roles, one gate each

Layer	AI drafts	Gate to pass	If it fails
L0 Comprehension	Rule records, fork classes	95% precision on review; every class, method and SQL statement mapped; every money rule reviewed	Re-extract with corrections; the module waits
L1 Transformation	C# translation, UI scaffolds	Compiler, tests and differential suite green; two reviewers on money code	Low acceptance for two sprints: engineers rebuild that module
L2 Migration	Column mappings, inferred keys	Every control total matches to the cent, per tenant	Tenant not promoted; legacy stays primary

Layer	AI drafts	Gate to pass	If it fails
L3 Equivalence	Test cases, diff clusters	100% equivalence on ledger, VAT and payroll; no unsigned deviations	Release blocked

L0: comprehension and rule inventory

- **Parse first.** A Roslyn parser maps 377 CSLA classes, 425 data-portal methods, 797 rule registrations and all 770 SQL statements. AI then fills in that map.
- **One record per rule.** AI writes each rule's statement, source lines, inputs and effective dates in two independent passes. A mechanical check rejects any record whose cited lines don't exist.
- **Forks and tickets too.** Fork diffs and 12 months of tickets are clustered.
- **People review the money rules.** The four legacy engineers and an accountant review every one. Module pods take over those engineers' maintenance work from week 2.

L1: translate the kernel, rebuild the shell

Module	Lines	Approach
Ledger	17,035	Translate posting and closing
Invoicing and VAT	24,704	Translate the calculation; the VAT return becomes a rule pack
Payables	11,475	Translate posting; rebuild intake around AI extraction
Bank import	29,408	Rebuild on standard ISO 20022 libraries; most legacy code here is generated
Inventory	38,208	Translate FIFO, LIFO and average costing
Fixed assets	29,967	Translate depreciation
Payroll	19,660	Translate the kernel; rates and 20 year-specific rules move to rule packs
Statements and e-filing	57,193	Rebuild on typed report models and rule packs
Desktop UI	180,484	Rebuild as the web client from AI-extracted screen specs

Every AI change is one reviewed pull request that has passed all checks. The share accepted without major rework is tracked per module to steer the method. It is never a quota for reviewers.

L2: schema, tenant keys and migration

- **Target schema.** The tenant is in every key, foreign keys are real, and money uses exact decimals at each field's legacy precision.
- **AI drafts, pipelines move.** AI drafts the mappings and inferred keys. Deterministic, restartable pipelines move the data.
- **Reconciliation to the cent:** trial balance, VAT by rate and period, open items, stock, assets and payroll year to date. Key reports are compared on both sides.
- **Weekly fleet rehearsal.** From month 4, all 4,200 databases are migrated in rehearsal every week, inside the production security boundary.
- **History is never recalculated.** It moves exactly as legacy computed it.

L3: proving financial equivalence

- **Same inputs, both systems.** Legacy runs headless as the oracle (A12). Both systems get the same inputs, and ledger lines, VAT lines, payslips and statements are compared.
- **Four test sets:**
 - replayed history from 300 tenants;
 - a million generated edge cases a night;
 - official golden files;
 - one test per rule.
- **Tolerance policy:**
 - noise below the declared precision is ignored;
 - half-cent boundary cases are approved once, as a class;
 - logic differences are fixed, or signed off one by one.
- **Mutation tests** flip a sign or a rate on purpose, to prove the harness catches it.
- **Shadow payrolls.** Two must match before payroll customers move.

AI in the product

Four AI features ship with the product. They draft, flag and explain. Only bounded, reversible, audited actions run without a person, and nothing AI-driven files a return, pays a salary or changes a closed period.

Feature	May act alone	Needs a person	Launch gate
Invoice and bank statement extraction	Post a purchase invoice when the tenant opted in, the supplier is known, all checks pass, confidence is 0.98 or more and the amount is under the tenant's limit; one-click reversal	New suppliers, low confidence, fuzzy bank matches	97% field accuracy
Ledger anomaly detection	Flag and explain	Acknowledge a high-severity flag before filing	60% useful flags
Natural-language reporting	Run read-only queries within the user's rights	Save or share a report	95% exact answers on 500 questions
Grounded support assistant	Answer with citations; open the right screen	Any change, offered as a button	90% grounded answers

How the guardrails work:

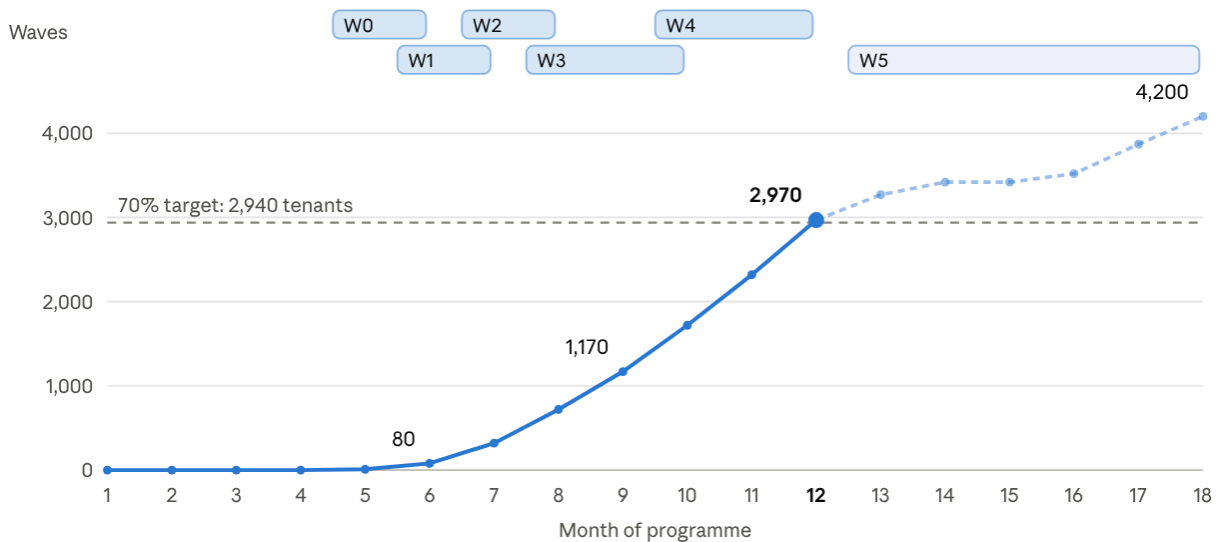
- **Extraction checks first.** Deterministic checks (VAT arithmetic, EU VAT-number lookup, duplicates, open period) run before anyone sees a draft.
- **Reporting numbers never come from the model.** The model never writes SQL: it picks metrics from a finance-owned catalogue, the engine runs the query, and every number comes from the result.
- **The assistant stays grounded.** It answers only from the manual, release notes and resolved tickets, and hands over to a person with a summary. Target: 30 percent of contacts resolved without an agent.
- **Data protection.** Models are EU-hosted behind a gateway, with no training on customer data (A13). Audit records are kept with the ledger, and every feature has a kill switch per tenant.

Migration plan

2,970 customers (70.7%) move in five waves by month 12. The last 1,230 follow by month 18, and legacy is switched off in month 20. Each customer moves overnight, and only one system may write its books at any moment.

2,970 tenants are live by month 12, just over the 70% target

Planned cumulative tenants migrated by month; wave W5 moves the last 1,230 by month 18



Wave plan in this document · cumulative tenants by month, months 1 to 18

Wave	Months	Tenants	Who moves
W0	5 to 6	20	The firm's own books and 19 volunteers
W1	6 to 7	150	Simple: no payroll, inventory or fork
W2	7 to 8	400	Adds payroll, after two matching shadow payrolls
W3	8 to 10	1,000	Inventory and assets; bureau portfolios together
W4	10 to 12	1,400	Forks mapped to extension manifests; dedicated databases
W5	13 to 18	1,230	The long tail and late adopters

Ordering rules:

- Lowest complexity first.
- A tenant moves only when every module it uses has passed L3.
- No cutovers from the 10th to the 25th (filing deadlines), or from 15 December to 15 February.
- Bureaus move as a portfolio.

The factory handles 100 tenants a night on about 10 usable nights a month. That gives roughly 1,000 a month against a planned peak of 650.

Coexistence: one writer per customer

- **One system of record.** A customer's books are written in legacy or in the cloud, never both.
- **An authority stamp** in the tenant registry says which system may write. Workers and connectors reject any command carrying an old stamp.
- **Legacy writers are locked out at cutover, not asked to stop.** Desktop users' MySQL grants are revoked, integrations are disabled, and the database is set read-only.
- **One sign-in portal** routes each user to the right system for each company.

Cutover runbook (per customer)

1. **T-14 days:** pre-flight scan (schema version, data quality, extension manifest). The customer is booked into training.
2. **T-7:** full rehearsal into staging. Every control total is reconciled, and the round trip back to legacy is proven lossless.
3. **T-7 to T-2:** the customer checks opening balances in a preview.
4. **T0, 20:00:** stamp and lock the legacy writers; final extract.
5. **T0, about 21:00:** load, reconcile, smoke tests, isolation probe. The stamp switches to the cloud.
6. **T+1 to T+35:** hypercare until the first filing from the cloud is accepted. The customer counts as migrated after five stable business days.

Change management:

- In-app tours, plus a short live session per role.
- A one-page sheet mapping desktop keys to the web client.
- A champion in every bureau.
- Early movers keep their price for 12 months.

Validation gates

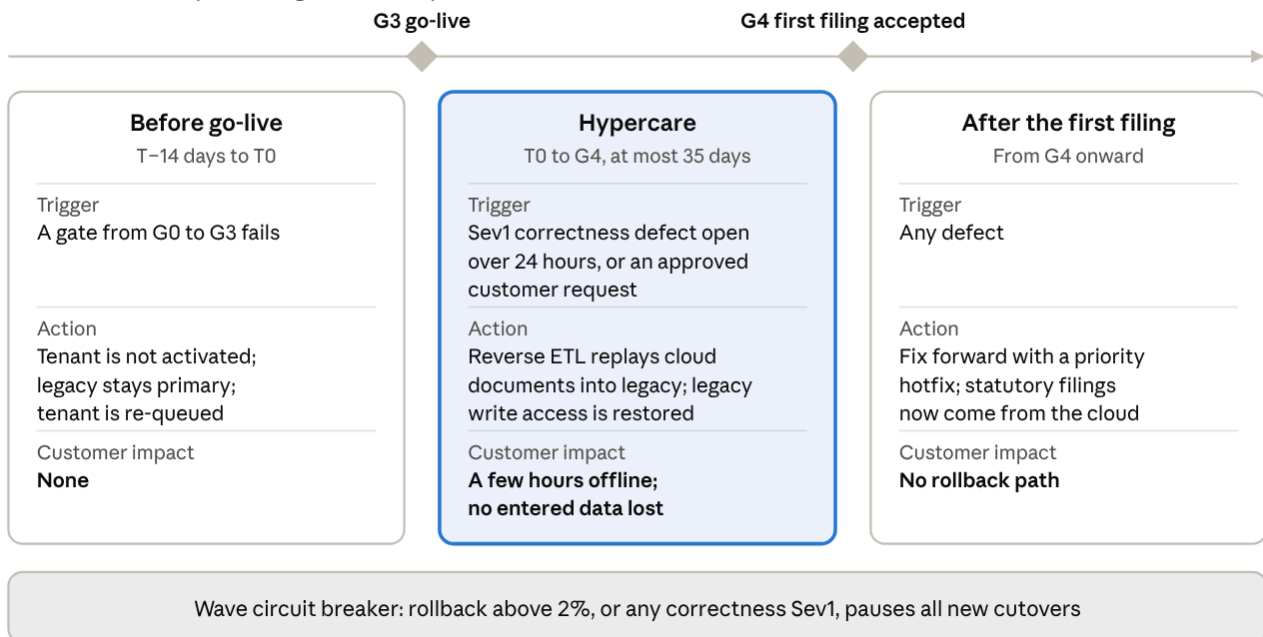
Gate	Passes when
G0 Eligibility	Every module used has passed L3; manifest complete; data fixed; payroll tenants have one matching shadow payroll
G1 Rehearsal	Every control total matches to the cent; the round trip is lossless

Gate	Passes when
G2 Acceptance	The customer or their bureau confirms opening balances
G3 Go-live	Final reconciliation matches; smoke tests and isolation probe clean
G4 Hypercare exit	First filings accepted, or day 35 if none is due; no Sev1 or Sev2 open; tickets at or below the legacy baseline
Wave gate	The previous wave's first 20%, live 30+ days, shows rollback under 2% and no correctness Sev1; the next wave's modules pass L3 at 100%

Rollback

Rollback is free before go-live and closes at the first filing

Per-tenant rollback paths along the cutover; phases not to scale



Rollback paths · three phases of a tenant cutover

The reverse path is rehearsed at every G1, so it works when needed. Once the tax authority accepts a filing produced in the cloud, defects are fixed forward.

Legacy end of life:

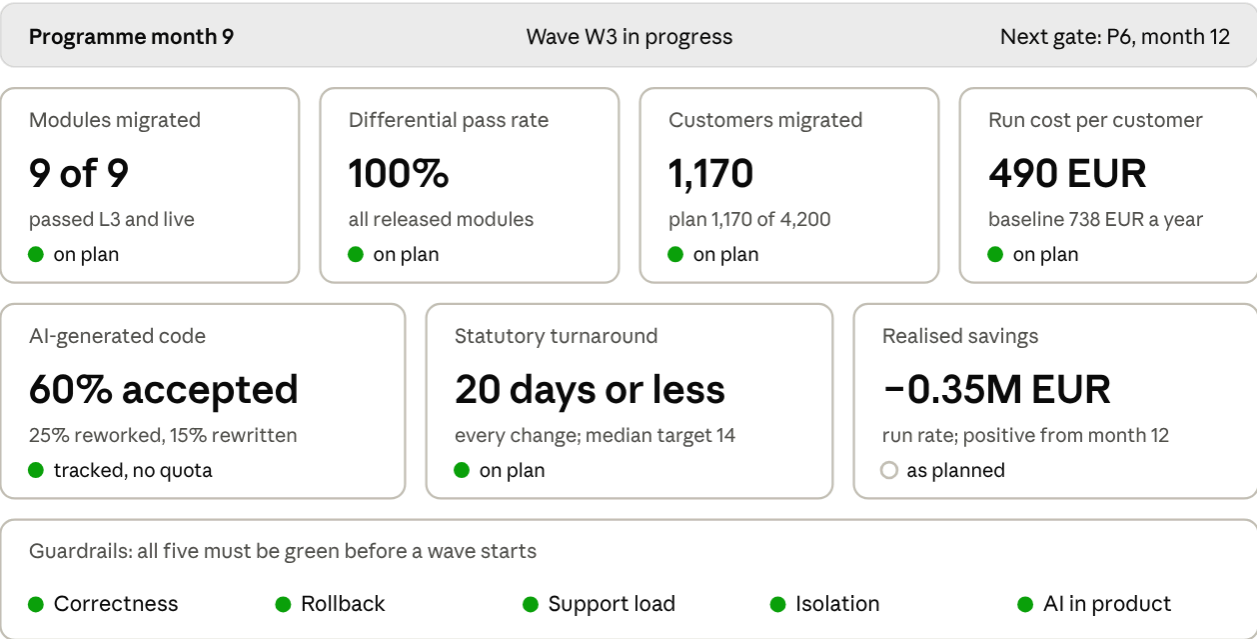
Month	Milestone
3	Forks frozen
6	Features frozen
9	End of life announced
18	Last statutory update
20	Hosting off; databases kept as encrypted, read-only archives for 10 years (A8)

KPI dashboard

Seven KPIs and five guardrails, refreshed weekly from live data and reviewed monthly by the steering committee. Correctness and isolation always outrank speed.

One page tells the steering committee whether to start the next wave

Mock-up showing month 9 plan values; every tile is a live query



KPI	How it is measured	Month 12 target
Modules migrated	Modules out of 9 that passed L3 and run for customers	9 of 9
Differential test pass rate	Equivalent cases over all cases, per module	100%; no unsigned deviations

KPI	How it is measured	Month 12 target
AI code accepted versus reworked	Share of AI changes merged without major rework, per module	Tracked to steer the method; no quota
Customers migrated against plan	Tenants past G3 and stable for five business days	2,970
Run cost per customer	Cloud run cost over cloud tenants; blended cost shown alongside	406 EUR or less (model: 330)
Statutory turnaround	Days from official publication to production	20 days at most; median 14
Realised savings	3.1M EUR baseline minus actual run cost, plus avoided support cost	+0.19M EUR run rate; 2.2M EUR from month 21

Guardrails. Any breach pauses new cutovers:

- zero correctness incidents;
- rollback under 2 percent;
- tickets per tenant at or below 1.55 a month;
- every isolation probe passes;
- AI auto-post errors under 0.5 percent.

Month-by-month targets are in the Reference tab.

Trade-off analysis

Each choice favours financial correctness and one code base over speed of the first release.

Decision	We chose	Over the alternative, because
Strategy	AI-assisted rebuild, moved tenant by tenant	Rehosting keeps 4,200 databases and slow tax changes; a manual rewrite doesn't fit 12 months
Tenancy	Pooled PostgreSQL in cells; dedicated for the largest	One database per customer keeps cost and upgrades per customer

Decision	We chose	Over the alternative, because
Architecture	Modular monolith	Microservices split one posting across services and need a bigger team to run
Database	PostgreSQL	MySQL has no row-level security; SQL Server adds licence cost per cell
Language and UI	C# and React	VB.NET is frozen; Blazor Server sends every keystroke over the network
Statutory	Rule packs plus reviewed code	Faster releases alone still need an engineer per rate change; a scripting engine is hard to govern
Forks	Three extension tiers	Per-customer branches bring back 310 builds; in-process plug-ins can reach other tenants' data
Migration	Waves, one writer per customer	Big-bang risks everyone at once; two-way sync lets two sets of books drift
AI	AI drafts, tools check, people sign	Unchecked AI output in money code has no accountable owner

Business impact

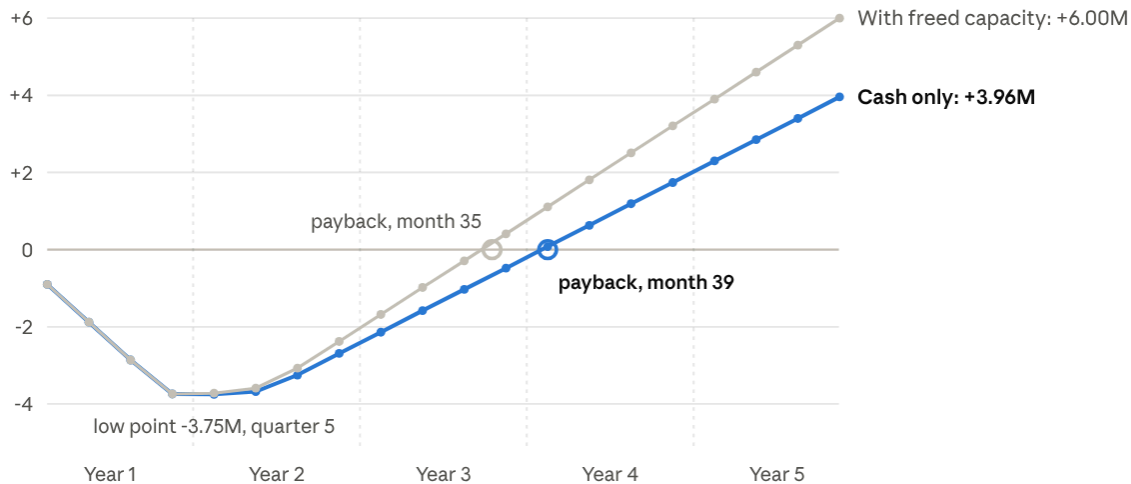
About 3.95M EUR over two years buys 2.2M EUR a year in cash savings from year 3. Payback comes in month 39 on cash, or month 35 counting freed engineering time, and five-year net cash is 3.96M EUR. The figures are indicative and are re-baselined in week 4.

Million EUR	Year 1	Year 2	Year 3	Years 4 and 5, each
Investment	3.51	0.44	0	0
Infrastructure saving against 3.1	-0.26	1.16	1.84	1.84
Support saving	0.03	0.33	0.37	0.37
Net cash	-3.74	1.05	2.22	2.22

Investment covers the delivery team, tooling and 15 percent contingency. The firm's existing staff are not counted, and freed engineering time is shown separately, not as cash. The inputs are in the Reference tab.

The programme pays back in month 39 on cash, month 35 counting freed capacity

Cumulative net benefit in million EUR, by quarter, base case



Business model in this document · quarterly cumulative net benefit, years 1 to 5

Sensitivity:

- Cloud costs 35 percent higher, landing exactly on the 406 EUR target: payback in month 47.
- Waves slip three months: payback in month 44.
- Team costs 25 percent more: payback in month 44.

Beyond the cash:

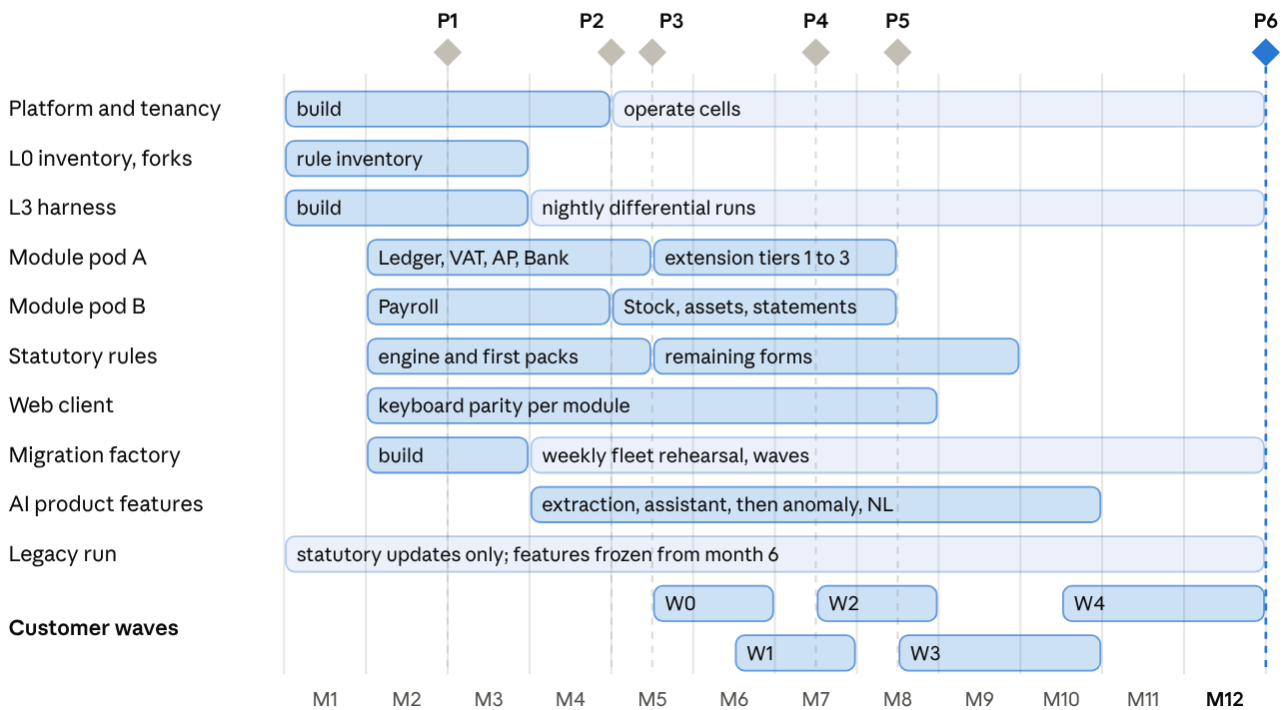
- Statutory changes stop missing the legal window.
- Maintenance falls from 62 percent of engineering time to 30 percent or less, putting about 10 engineers back on the roadmap.
- Tickets fall about 40 percent, from 6,500 to 3,900 a month.
- A new customer adds about 230 EUR a year in run cost, against 590 EUR on legacy.

Roadmap, team and risks

Ten workstreams run in parallel through six programme gates. The first customers go live in month 5, and the 70 percent mark is reached in month 12.

First customers in month 5, 70% by month 12, through six programme gates

Workstreams by month; darker bars build, lighter bars run; diamonds are programme gates P1 to P6



12-month roadmap · 10 workstreams, 5 waves, 6 programme gates

Gate	When	Passes when
P1 Baseline	Week 8	Assumptions confirmed; L0 at 95% precision on Ledger and VAT; legacy harness running
P2 Platform ready	Month 4	Two cells live; isolation probes and penetration test passed; fleet rehearsal running
P3 First customers	Mid month 5	Ledger, Invoicing and VAT, Payables, Bank import and VAT filing pass L3; keyboard parity met
P4 Payroll	Mid month 7	Shadow payrolls for months 5 and 6 match; Sodra and income-tax packs accepted in test
P5 Full scope	Mid month 8	All nine modules pass L3; extension tiers live
P6 70 percent	Month 12	2,940 or more tenants migrated; guardrails green

Team

A delivery team of 32 works for 12 months, then 10 for months 13 to 18. It works in pods alongside about a dozen of the firm's own people, so the firm can run the platform alone afterwards.

Pod	People	Owns
Programme lead	1	Architecture, gates, steering
Platform and SRE	7	Cells, tenancy, identity, CI/CD
Module pods A and B	14	The nine business modules
Web client	6	Keyboard-first client, parity tests
Migration and equivalence	9	Pipeline, reconciliation, L3 harness
AI and statutory	7	L0 and L1 tooling, product AI, rule packs
Domain authority (the firm)	4 legacy engineers, accountants, product owner	Rule review, deviation sign-off

Top risks

Risk	Mitigation
Knowledge sits with four engineers	L0 inventory; their review time protected; pods take over their maintenance from week 2
Hidden calculation differences	Four L3 test sets, mutation tests, tolerance policy, deviation register
Accountants reject the web client	Speed-parity gate, classic key map, bureau champions
Forks don't fit the tiers	All 310 classified by week 6; misfits move to W5
Data quality across 4,200 databases	Weekly fleet rehearsal from month 4
Cross-tenant data exposure	Forced row-level security, isolation beyond the database, hourly probes, penetration tests

Code evidence, the full technology stack, cost inputs, month-by-month KPI targets and a glossary are in the [Reference](#) tab.